In [16]:

```python
import numpy as np
import matplotlib.pyplot as plt
import os
import matplotlib.gridspec as gridspec


def generate_signals(n_samples=10000, fs=8000,
                     signal_freq=300.0, noise_std=0.5, seed=42):

    rng = np.random.default_rng(seed)
    t = np.arange(n_samples) / fs

    clean = np.sin(2 * np.pi * signal_freq * t)

    primary_noise = rng.normal(0, noise_std, n_samples)
    ref_noise = 0.9 * np.roll(primary_noise, 2)

    noisy = clean + primary_noise
    return t, clean, ref_noise, noisy


def lms(d, x, filter_len=10, mu=0.005):

    n = len(d)
    w = np.zeros(filter_len)
    e = np.zeros(n)
    mse = np.zeros(n)

    for i in range(filter_len, n):
        x_vec = x[i-filter_len:i][::-1]
        y = np.dot(w, x_vec)
        e[i] = d[i] - y
        w = w + 2 * mu * e[i] * x_vec
        mse[i] = e[i] ** 2

    return e, mse

def nlms(d, x, filter_len=10, alpha=0.5, c=1e-6):

    n = len(d)
    w = np.zeros(filter_len)
    e = np.zeros(n)
    mse = np.zeros(n)

    for i in range(filter_len, n):
        x_vec = x[i-filter_len:i][::-1]
        y = np.dot(w, x_vec)
        e[i] = d[i] - y

        mu = alpha / (c + np.dot(x_vec, x_vec))
        w = w + mu * e[i] * x_vec
        mse[i] = e[i] ** 2
```

```python
    return e, mse

def rls(d, x, filter_len=10, lam=0.99, delta=1.0):

    n = len(d)
    w = np.zeros(filter_len)
    P = delta * np.eye(filter_len)

    e = np.zeros(n)
    mse = np.zeros(n)

    for i in range(filter_len, n):
        x_vec = x[i-filter_len:i][::-1]
        y = np.dot(w, x_vec)
        e[i] = d[i] - y

        Px = P @ x_vec
        k = Px / (lam + x_vec @ Px)

        w = w + k * e[i]
        P = (P - np.outer(k, Px)) / lam

        mse[i] = e[i] ** 2

    return e, mse

def modified_nlms(d, x, filter_len=10, alpha=0.5, c=1e-6):

    n = len(d)
    w = np.zeros(filter_len)
    e = np.zeros(n)
    mse = np.zeros(n)

    for i in range(filter_len, n):
        x_vec = x[i-filter_len:i][::-1]
        y = np.dot(w, x_vec)
        e[i] = d[i] - y

        mu = alpha / (c + np.dot(x_vec, x_vec))
        sign_e = np.sign(e[i])
        w = w + mu * sign_e * x_vec
        mse[i] = e[i] ** 2

    return e, mse

def compute_mse_db(mse, warmup=500):

    valid = mse[warmup:]
    valid = np.maximum(valid, 1e-12)
    return 10 * np.log10(np.mean(valid))


def log_spectral_distance(clean, enhanced, n_fft=512):
```

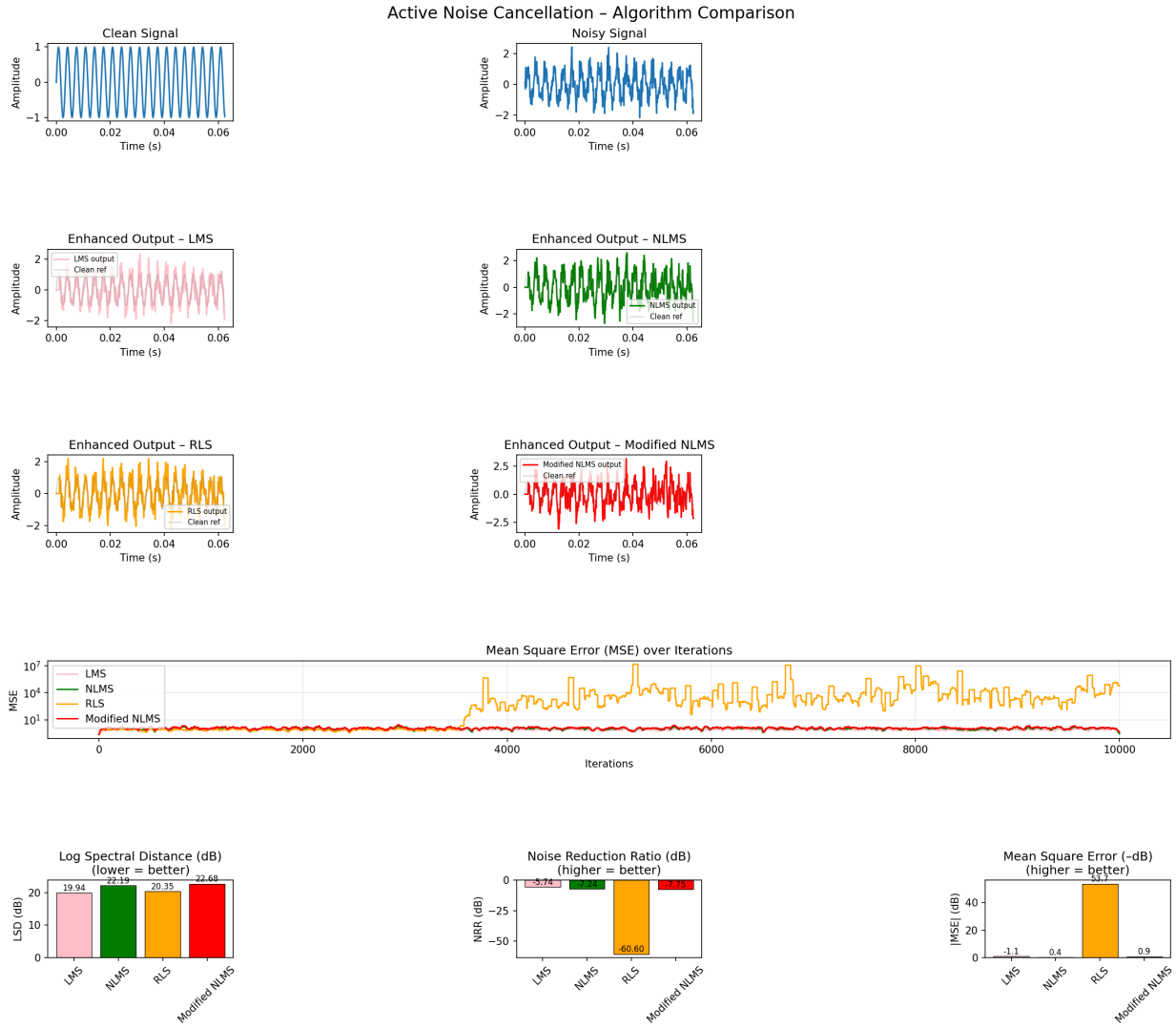
```
    eps = 1e-10
    P_clean = np.abs(np.fft.rfft(clean, n=n_fft))**2 + eps
    P_enh = np.abs(np.fft.rfft(enhanced, n=n_fft))**2 + eps

    return np.sqrt(np.mean((10 * np.log10(P_enh / P_clean))**2))


def noise_reduction_ratio(ref_noise, error, warmup=500):

    sigma_x2 = np.mean(ref_noise[warmup:]**2)
    sigma_e2 = np.mean(error[warmup:]**2)
    sigma_e2 = max(sigma_e2, 1e-12)

    return 10 * np.log10(sigma_x2 / sigma_e2)


def plot_results(t, clean, noisy, results, n_show=500):

    os.makedirs("outputs", exist_ok=True)

    fig = plt.figure(figsize=(16, 14), constrained_layout=True)
    fig.suptitle("Active Noise Cancellation — Algorithm Comparison", fontsize=

    colors = {"LMS": "pink", "NLMS": "green",
              "RLS": "orange", "Modified NLMS": "red"}


    gs = gridspec.GridSpec(5, 3, figure=fig, hspace=0.4, wspace=0.3)


    ax1 = fig.add_subplot(gs[0, 0])
    ax1.plot(t[:n_show], clean[:n_show])
    ax1.set_title("Clean Signal")
    ax1.set_xlabel("Time (s)"); ax1.set_ylabel("Amplitude")

    ax2 = fig.add_subplot(gs[0, 1])
    ax2.plot(t[:n_show], noisy[:n_show])
    ax2.set_title("Noisy Signal")
    ax2.set_xlabel("Time (s)"); ax2.set_ylabel("Amplitude")

    ax_dummy_0_2 = fig.add_subplot(gs[0, 2])
    ax_dummy_0_2.axis('off')

    output_axes = [
        fig.add_subplot(gs[1, 0]),
        fig.add_subplot(gs[1, 1]),
        fig.add_subplot(gs[2, 0]),
        fig.add_subplot(gs[2, 1])
    ]
    outputs_list = list(results.items())
    for i, (name, data) in enumerate(outputs_list):
        ax = output_axes[i]
```

```
        ax.plot(t[:n_show], data["error"][:n_show], label=f"{name} output", co
        ax.plot(t[:n_show], clean[:n_show], color="gray", alpha=0.4, linewidth
        ax.set_title(f"Enhanced Output — {name}")
        ax.set_xlabel("Time (s)"); ax.set_ylabel("Amplitude")
        ax.legend(fontsize=7)


    ax_dummy_1_2 = fig.add_subplot(gs[1, 2])
    ax_dummy_1_2.axis('off')
    ax_dummy_2_2 = fig.add_subplot(gs[2, 2])
    ax_dummy_2_2.axis('off')

    ax_mse = fig.add_subplot(gs[3, :])
    for name, data in results.items():
        smooth = np.convolve(data["mse"], np.ones(50)/50, mode='same')
        ax_mse.semilogy(smooth, label=name, color=colors[name], linewidth=1.4)
    ax_mse.set_title("Mean Square Error (MSE) over Iterations")
    ax_mse.set_xlabel("Iterations"); ax_mse.set_ylabel("MSE")
    ax_mse.legend(); ax_mse.grid(True, alpha=0.3)

    names = list(results.keys())
    bar_colors = [colors[n] for n in names]

    ax_lsd = fig.add_subplot(gs[4, 0])
    lsd_vals = [results[n]["lsd"] for n in names]
    bars = ax_lsd.bar(names, lsd_vals, color=bar_colors, edgecolor="black", li
    ax_lsd.set_title("Log Spectral Distance (dB)\n(lower = better)")
    ax_lsd.set_ylabel("LSD (dB)")
    ax_lsd.tick_params(axis='x', rotation=45)
    for bar, val in zip(bars, lsd_vals):
        ax_lsd.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.02,
                    f"{val:.2f}", ha='center', va='bottom', fontsize=8)


    ax_nrr = fig.add_subplot(gs[4, 1])
    nrr_vals = [results[n]["nrr"] for n in names]
    bars = ax_nrr.bar(names, nrr_vals, color=bar_colors, edgecolor="black", li
    ax_nrr.set_title("Noise Reduction Ratio (dB)\n(higher = better)")
    ax_nrr.set_ylabel("NRR (dB)")
    ax_nrr.tick_params(axis='x', rotation=45)
    for bar, val in zip(bars, nrr_vals):
        ax_nrr.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.05,
                    f"{val:.2f}", ha='center', va='bottom', fontsize=8)


    ax_mse_bar = fig.add_subplot(gs[4, 2])
    mse_vals = [results[n]["mse_db"] for n in names]
    bars = ax_mse_bar.bar(names, [abs(v) for v in mse_vals],
                          color=bar_colors, edgecolor="black", linewidth=0.6)
    ax_mse_bar.set_title("Mean Square Error (−dB)\n(higher = better)")
    ax_mse_bar.set_ylabel("|MSE| (dB)")
    ax_mse_bar.tick_params(axis='x', rotation=45)
    for bar, val in zip(bars, mse_vals):
```

```python
        ax_mse_bar.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.
                        f"{val:.1f}", ha='center', va='bottom', fontsize=8)

    out_path = "outputs/anc_results.png"
    plt.savefig(out_path, dpi=150)
    print(f"Plot saved → {out_path}")
    plt.close(fig)

def main():

    t, clean, ref_noise, noisy = generate_signals()

    e_lms, mse_lms = lms(noisy, ref_noise)
    e_nlms, mse_nlms = nlms(noisy, ref_noise)
    e_rls, mse_rls = rls(noisy, ref_noise)
    e_mod, mse_mod = modified_nlms(noisy, ref_noise)

    results = {
        "LMS": {"error": e_lms, "mse": mse_lms},
        "NLMS": {"error": e_nlms, "mse": mse_nlms},
        "RLS": {"error": e_rls, "mse": mse_rls},
        "Modified NLMS": {"error": e_mod, "mse": mse_mod},
    }

    for name, data in results.items():
        data["mse_db"] = compute_mse_db(data["mse"])
        data["lsd"] = log_spectral_distance(clean, data["error"])
        data["nrr"] = noise_reduction_ratio(ref_noise, data["error"])

    plot_results(t, clean, noisy, results)


if __name__ == "__main__":
    main()

    from IPython.display import Image
Image("outputs/anc_results.png")
```

```
Plot saved → outputs/anc_results.png
```

Out[16]:



Active Noise Cancellation (ANC) using Adaptive Filters

This project is a simple implementation of **Active Noise Cancellation (ANC)** using adaptive filtering techniques in Python. The main idea is to recover a clean signal from a noisy input and compare how different adaptive algorithms perform in doing so.

Overview

In practical audio systems, signals are often affected by unwanted noise. This project simulates a **feedforward ANC system**, where:

- A **primary input** contains the original signal mixed with noise

- A **reference input** contains noise that is correlated with the primary noise
- An **adaptive filter** learns to estimate and remove this noise

The final output is a cleaner version of the original signal.

Algorithms Implemented

The following adaptive algorithms are implemented and compared:

- Least Mean Square (**LMS**)
- Normalized Least Mean Square (**NLMS**)
- Recursive Least Square (**RLS**)
- Modified NLMS (**sign-based variant**)

How It Works

The noisy signal is modeled as:

```
d(n) = x(n) + n₂(n)
```

Where:

- `x(n)` → clean signal
- `n₂(n)` → noise

A correlated noise reference `n₁(n)` is passed through an adaptive filter:

```
e(n) = d(n) - y(n)
```

Where:

- `y(n)` → estimated noise
- `e(n)` → final output (ideally close to the clean signal)

The adaptive algorithm continuously updates the filter to minimize this error.

Performance Metrics

To compare the algorithms, the following metrics are used:

- **Mean Square Error (MSE)** → measures how small the error is (lower is better)
- **Log Spectral Distance (LSD)** → compares frequency content (lower is better)
- **Noise Reduction Ratio (NRR)** → shows how much noise is removed (higher is better)

Output

The program generates:

- Plots of clean and noisy signals
- Filtered outputs for each algorithm
- MSE convergence curves
- Comparison charts for LSD, NRR, and MSE

Output image:

```
outputs/anc_results.png
```

Requirements

Install the required libraries:

```
pip install numpy matplotlib
```

How to Run

```
python main.py
```

Key Observations

- **LMS** → simple and easy to implement, but converges slowly

- **NLMS** → improves stability and convergence compared to LMS
- **RLS** → very fast convergence, but computationally expensive
- **Modified NLMS** → offers a good balance between speed and complexity

---

Reference / Citation

This implementation is inspired by the following paper:

> Atar Mon, Thiri Thandar Aung, Chit Htay Lwin "*Active Noise Cancellation in Audio Signal Processing*" International Research Journal of Engineering and Technology (IRJET), 2016

BibTeX

```
@article{mon2016anc,
  title={Active Noise Cancellation in Audio Signal Processing},
  author={Mon, Atar and Aung, Thiri Thandar and Lwin, Chit Htay},
  journal={International Research Journal of Engineering and
Technology (IRJET)},
  volume={3},
  number={11},
  year={2016}
}
```

Notes

- This is a **simulation-based project** (not real-time ANC)
- Noise used: Gaussian noise
- Signal used: sinusoidal waveform

Future Improvements

- Real-time ANC using microphone input
- Using speech signals instead of sine waves
- Implementation on embedded systems or DSP hardware

Submitted From

KUNCHUR KOTRESHA , 241EC130 , NATIONAL INSTITUTE OF TECNOLOGY KARNATAKA,SURATHKAL

If you found this useful

Feel free to the repo and share your feedback!

In [ ]: